Performance Evaluation of FlashAttention-2 for Inference on CPU

Yu-Ting Lee [1]

Abstract

FlashAttention has revolutionized transformer training and inference on GPU by minimizing memory I/O, yet its behavior on CPU remains underexplored. We evaluate the FlashAttention-2 (FA-2) forward kernel for inference on a single CPU against standard attention and PyTorch's scaled dot-product attention (SDPA). For masked attention, we compare three scheduling strategies for parallelizing the kernel. We further profile the runtime breakdown of the kernel's algorithmic steps. For self-attention, FA-2 performs on par with SDPA; for masked attention, it is 1.4× to 1.7× faster than SDPA on long sequences. Per-thread profiling shows that static scheduling leaves 45% of thread time idle on the triangular masked workload; dynamic and work-stealing scheduling reduce it to 2–5% and speed up the kernel by up to 1.4×. Our analysis serves as a preliminary exploration of efficient attention computations in resource-constrained settings, including CPU-only inference.

1. Introduction

The attention mechanism is the core of transformer models (Vaswani et al., 2017), driving their success across various tasks. However, standard attention incurs quadratic computational and I/O complexity with respect to sequence length. As the sequence length grows, the intermediate results of attention often exceed the capacity of fast on-chip memory (SRAM or cache), and the resulting traffic to off-chip memory becomes a major performance bottleneck.

To address this bottleneck, FlashAttention-2 (FA-2) (Dao, 2024) was introduced as a hardware-aware, exact attention algorithm. Through tiling and kernel fusion, it computes attention in a single pass without materializing the large intermediate matrices. While FA-2 has been extensively optimized and adopted for GPU, its performance on general-purpose CPU remains largely unexplored. This gap is significant because CPU inference remains critical for resource-constrained environments (Jiang et al., 2023), such as edge computing, where high-end GPUs are often unavailable.

In this work, we investigate the efficacy of the FA-2 forward kernel for inference on a single CPU. We benchmark multi-threaded implementations of the forward kernel against standard attention and PyTorch's scaled dot-product attention (SDPA) across a range of sequence lengths, head dimensions, and block sizes. For masked attention, we further evaluate three threading strategies—static, centralized dynamic, and work-stealing—for parallelizing the kernel. We also profile the runtime distribution of the kernel's algorithmic steps.

Our results show that, with suitable threading and block sizes, FA-2 performs on par with SDPA for self-attention and is 1.39–1.69× faster for masked attention on long sequences. Under masking, static scheduling leaves 45% of thread time idle; dynamic and work-stealing scheduling reduce it to 2–5% and are up to 1.43× faster than static scheduling. Our code is available for reproducibility.[1]

2. Background

2.1. Efficient Attention Computations

To compute attention efficiently on long contexts, numerous methods have been proposed to approximate attention (Beltagy et al., 2020; Chen et al., 2021; Choromanski et al., 2021; Katharopoulos et al., 2020; Kitaev et al., 2020; Roy et al., 2021; Zaheer et al., 2020). These encompass sparse attention patterns (Beltagy et al., 2020; Kitaev et al., 2020; Roy et al., 2021; Zaheer et al., 2020), low-rank approximations (Choromanski et al., 2021; Katharopoulos et al., 2020), and hybrid approaches (Chen et al., 2021). On CPU specifically, IceFormer (Mao et al., 2024) and NoMAD-attention (Zhang et al., 2024) accelerate attention inference via approximate methods, using k-nearest-neighbor search and SIMD in-register lookups, respectively. However, these approximation methods typically trade model accuracy for theoretical efficiency and may fail to achieve true wall-clock speedups against standard attention.

[1]Graduate Institute of Communication Engineering, National Taiwan University, Taipei, Taiwan. Correspondence to: Yu-Ting Lee <r14942088@ntu.edu.tw>.

Independent Project.

[1]https://github.com/Yu-TingLee/flash-attention2-cpu

In contrast, exact attention methods compute the mathematically equivalent attention matrix while optimizing data movement. Specifically, FlashAttention (Dao et al., 2022) and FlashAttention-2 (Dao, 2024) demonstrated that attention is bottlenecked by I/O cost, and attention computations can be substantially faster on GPU by minimizing memory accesses. Crucially, whether these hardware-aware advantages translate to CPU remains an important open question.

2.2. Hardware Characteristics: CPU vs. GPU

2.2.1. Performance Characteristics

GPU consists of compute elements (e.g., CUDA cores) and a memory hierarchy. The memory hierarchy comprises high bandwidth memory (HBM) and on-chip SRAM. For example, the A100 GPU has 40-80 GB of HBM with bandwidth 1.5-2.0 TB/s and 192 KB of on-chip SRAM per streaming multiprocessor (108 total) with bandwidth estimated around 19 TB/s (Dao et al., 2022). CPU relies on a multi-level cache hierarchy (L1, L2, L3) backed by off-chip DRAM, where caches are implicitly managed by hardware. As a reference, the i9-11900K used in our experiments has 512 KiB of L2 cache per core and 16 MiB of shared L3 cache, with a peak DRAM bandwidth of approximately 38.4 GB/s.

2.2.2. Execution Model

GPU achieves massive parallelism via the SIMT execution model, where threads execute in groups of 32 (warps) across streaming multiprocessors. Modern GPU also features specialized matrix multiply units (e.g., Tensor Cores) that can deliver over an order of magnitude higher throughput than general-purpose floating-point units. For instance, the A100 achieves 312 TFLOPs/s for FP16/BF16 matmul versus 19.5 TFLOPs/s for non-matmul FP32. In contrast, CPU relies on a smaller number of cores optimized for complex instruction streams, with parallelism expressed through both SIMD vector units and thread-level concurrency.

3. A Comparison between Standard Attention and the FA-2 Forward Kernel

Let $W_Q \in \mathbb{R}^{d\times d}, W_K \in \mathbb{R}^{d\times d}, W_V \in \mathbb{R}^{d\times d}$ be the trainable weights and $\widetilde{Z} \in \mathbb{R}^{T\times d}$ be the input matrix. For a transformer model, T specifies the input sequence length and d is the head dimension. The query, key, and value matrices are defined to be $\mathbf{Q} := \widetilde{Z}W_Q \in \mathbb{R}^{T\times d}, \mathbf{K} := \widetilde{Z}W_K \in \mathbb{R}^{T\times d}, \mathbf{V} := \widetilde{Z}W_V \in \mathbb{R}^{T\times d}$. The computation of attention can be summarized as

$$\mathbf{O} := \mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d})\mathbf{V} \in \mathbb{R}^{T\times d} \tag{1}$$

where $\mathbf{O}$ is the output matrix and the softmax is applied row-wise. See Figure. 1 for a visualization of standard attention.

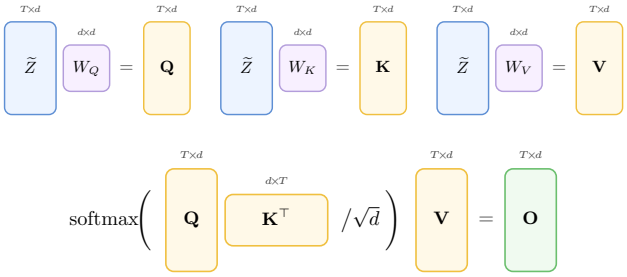


Figure 1. **Computation of standard attention.** Input matrix $\widetilde{Z}$ is projected by trainable weights W_Q, W_K, and W_V to obtain the query matrix $\mathbf{Q}$, key matrix $\mathbf{K}$, and value matrix $\mathbf{V}$. Output matrix $\mathbf{O}$ is obtained by applying row-wise softmax to the scaled product $\mathbf{Q}\mathbf{K}^\top/\sqrt{d}$, then multiplying by $\mathbf{V}$.

It is well-known that the computational complexity of standard attention is $\mathcal{O}(T^2d)$, while the I/O complexity (memory accesses) is $\mathcal{O}(T^2)$, when the intermediate $T \times T$ matrices cannot fit in on-chip memory. Therefore, standard attention is less efficient for long-sequence workloads. For a detailed derivation, please see Sec. A. It is thus crucial to design I/O-efficient attention algorithms that reduce memory accesses while preserving computational equivalence to standard attention. Specifically, it is desirable that the high speed of on-chip SRAM can be fully utilized.

A natural approach is to partition $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ into row blocks $\mathbf{Q}_{I_1}, \ldots, \mathbf{Q}_{I_{T_r}}$, $\mathbf{K}_{J_1}, \ldots, \mathbf{K}_{J_{T_c}}$, and $\mathbf{V}_{J_1}, \ldots, \mathbf{V}_{J_{T_c}}$ that fit entirely in on-chip memory and to compute the corresponding output row blocks $\mathbf{O}_{I_1}, \ldots, \mathbf{O}_{I_{T_r}}$. This technique is commonly referred to as *tiling* in literature. Tiling alone, however, is not enough.

For example, a safe softmax naively requires three passes over the input: one to compute the row-wise maximum, one to compute the sum of exponentials, and one to compute the normalized output, and these redundant memory accesses degrade performance for long sequences. Moreover, the non-linearity of softmax prevents us from naively summing partial results across column blocks. We need to rescale existing partial output to account for the new normalization factor before adding the contribution from the current blocks of $\mathbf{K}^\top$ and $\mathbf{V}$. Resolving these issues requires either two passes over the data or, as in the FA-2 forward kernel, an *online update* that maintains running statistics and rescales partial outputs as new row-blocks of $\mathbf{K}_{J_j}^\top$ and $\mathbf{V}_{J_j}$ are processed (Sec. B).

The resulting FA-2 forward kernel is shown in Algorithm 1 with a visual illustration in Figure. 2. We can show that Algorithm 1 achieves the same computational complexity of $\mathcal{O}(T^2d)$ while reducing I/O complexity to $\mathcal{O}(\frac{T^2d^2}{M})$, where M is the on-chip memory budget (Sec. C). For typical values of T and d, this represents a significant reduction in I/O complexity compared to standard attention. A line-by-line

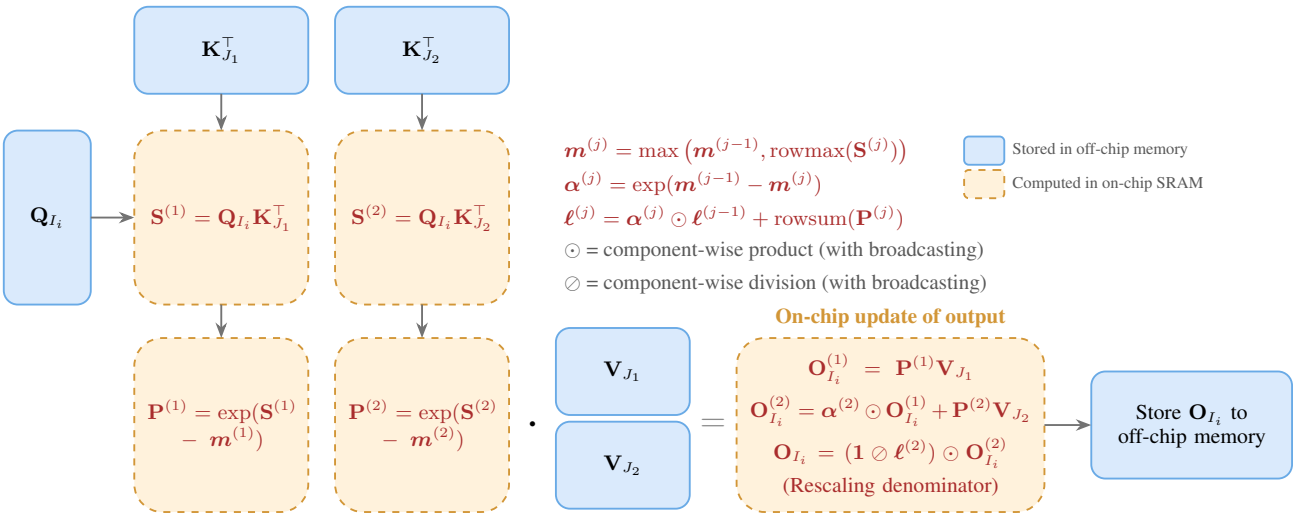


Figure 2. **Computation of the FA-2 forward kernel (Algorithm 1).** We assume only $T_c = 2$ column blocks. For a fixed row block I_i, the key $\mathbf{K}_{J_j}$ and value $\mathbf{V}_{J_j}$ blocks are loaded sequentially into on-chip SRAM, where the intermediate matrices $\mathbf{S}^{(j)}$, $\mathbf{P}^{(j)}$, and the running statistics $\boldsymbol{m}^{(j)}$, $\boldsymbol{\ell}^{(j)}$ are computed and used to update $\mathbf{O}_{I_i}$ in place. After all column blocks are processed, the output is rescaled by $1/\boldsymbol{\ell}$ and stored to off-chip memory. This avoids materializing the $T \times T$ intermediate matrices $\mathbf{S}$ and $\mathbf{P}$ in off-chip memory.

complexity analysis (Sec. C.3) further shows that Line 7 (Load $\mathbf{K}, \mathbf{V}$) dominates the memory accesses with $\mathcal{O}(\frac{T^2 d}{|I|})$ cost, while Line 8 (Compute $\mathbf{S}$) and Line 11 (Compute $\mathbf{O}$ in inner loop) dominate the computation complexity, both by the scale of $T^2 d$. This analysis also serves as the reference for our profiling experiments in Sec. 5.4.

4. Experimental Settings

Table 1. **CPU specifications.**

CPU Info	
Model Name	11th Gen Intel(R) Core(TM) i9-11900K @ 3.50GHz
Cores / Threads	8 Physical / 16 Logical
L2 Cache Size	512 KiB per core
L3 Cache Size	16 MiB (Shared)

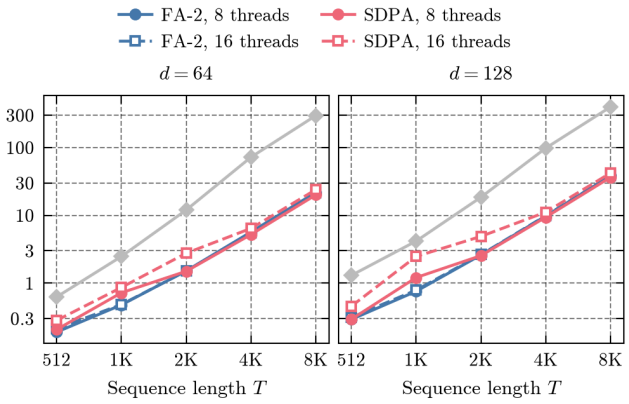
We evaluate the forward kernel in isolation using 50 reference test sets of synthetic inputs. The reference test sets contain inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ and ground-truth outputs $\mathbf{O}$ used to verify the correctness of all implementations. As end-to-end performance depends on broader system factors like KV cache management and memory allocation overhead, we leave further exploration to future work.

All implementations are written in C++ using Eigen 5.0.1 and are compiled with GCC 11 using `-O3`, `-march=native` (AVX-512), and `-ffast-math`. CPU specifications are summarized in Table 1.

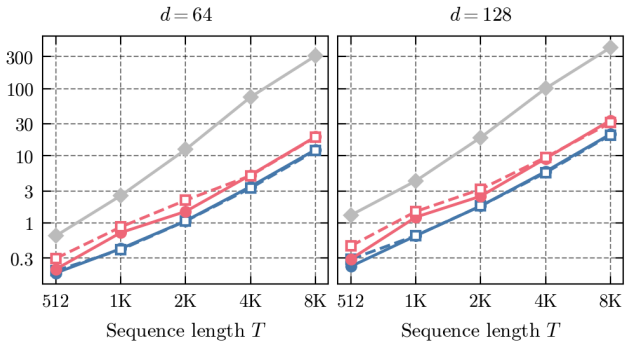
We evaluate FA-2 with 8 and 16 threads against SDPA with 8 and 16 threads and a single-threaded standard attention baseline. For masked attention (uneven workload), FA-2 skips key blocks above the diagonal, and we compare three scheduling strategies: (1) static, (2) centralized dynamic, and (3) work-stealing.

Algorithm 1 FA-2 Forward Kernel

1: Divide $\mathbf{Q}$ into row-block matrices $\mathbf{Q}_{I_1}, \dots \mathbf{Q}_{I_{T_r}}$ of size $|I| \times d$ and divide $\mathbf{K}, \mathbf{V}$ into $\mathbf{K}_{J_1}, \dots, \mathbf{K}_{J_{T_c}}$ and $\mathbf{V}_{J_1}, \dots, \mathbf{V}_{J_{T_c}}$ of size $|J| \times d$ each.
2: Divide the output $\mathbf{O}$ into row-block matrices $\mathbf{O}_{I_1}, \dots \mathbf{O}_{I_{T_r}}$ of size $|I| \times d$ each.
3: **for** $i = 1$ to T_r **do**
4: Load $\mathbf{Q}_{I_i}$ to on-chip SRAM.
5: Initialize $\mathbf{O}_{I_i} \leftarrow \mathbf{0}_{|I_i| \times d}$, $\boldsymbol{\ell} \leftarrow \mathbf{0}_{|I_i|}$ and $\boldsymbol{m} \leftarrow (-\infty)_{|I_i|}$ on chip.
6: **for** $j = 1$ to T_c **do**
7: Load $\mathbf{K}_{J_j}$ and $\mathbf{V}_{J_j}$ to on-chip SRAM.
8: Compute $\mathbf{S} \leftarrow \mathbf{Q}_{I_i}\mathbf{K}_{J_j}^\top$ on chip.
9: # $\oslash$ = component-wise division; $\odot$ = component-wise product
10: Compute $\boldsymbol{m}_{\text{new}} \leftarrow \max(\boldsymbol{m}, \text{rowmax}(\mathbf{S}))$, $\mathbf{P} \leftarrow \exp(\mathbf{S} - \boldsymbol{m}_{\text{new}})$, $\boldsymbol{\ell}_{\text{new}} \leftarrow \boldsymbol{\alpha} \odot \boldsymbol{\ell} + \text{rowsum}(\mathbf{P})$ on chip, where $\boldsymbol{\alpha} = \exp(\boldsymbol{m} - \boldsymbol{m}_{\text{new}})$.
11: Compute $\mathbf{O}_{I_i} \leftarrow \boldsymbol{\alpha} \odot \mathbf{O}_{I_i} + \mathbf{P}\mathbf{V}_{J_j}$ on chip.
12: Compute $\boldsymbol{m} \leftarrow \boldsymbol{m}_{\text{new}}, \boldsymbol{\ell} \leftarrow \boldsymbol{\ell}_{\text{new}}$ on chip.
13: **end for**
14: Compute $\mathbf{O}_{I_i} \leftarrow (\mathbf{1} \oslash \boldsymbol{\ell}) \odot \mathbf{O}_{I_i}$ on chip.
15: Store $\mathbf{O}_{I_i}$ to off-chip memory.
16: **end for**

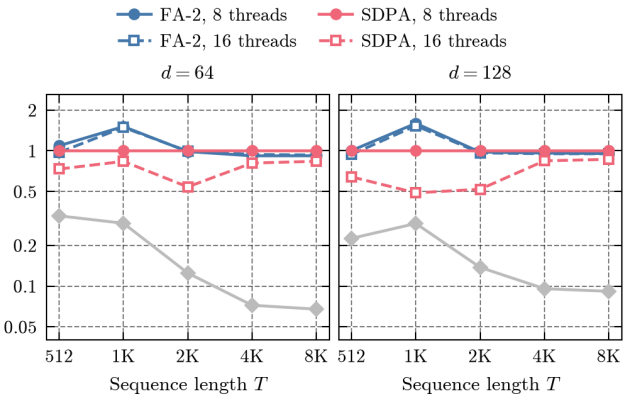


(a) Self-attention.

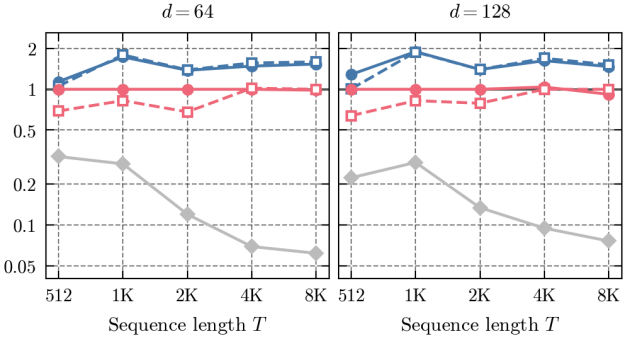


(b) Masked attention.

Figure 3. **Runtime of FA-2, PyTorch SDPA, and standard attention.** The y-axis is runtime in ms (log scale); each method uses its best configuration for each (T, d) (Sec. 4). The gray line is single-threaded standard attention. For self-attention, FA-2 is on par with SDPA for $T \geq 2048$ and faster at $T = 1024$. For masked attention, FA-2 is the fastest at every T.



(a) Self-attention.



(b) Masked attention.

Figure 4. **Speedup over PyTorch SDPA.** The y-axis (log scale) is the speedup relative to SDPA, where the SDPA baseline uses the faster of 8 and 16 threads for each (T, d). The gray line is single-threaded standard attention. For self-attention, FA-2 matches SDPA for $T \geq 2048$ (0.93–0.99×) and is up to 1.6× faster at $T = 1024$; for masked attention, FA-2 is 1.39–1.69× faster for $T \geq 2048$.

We measure the runtime of each implementation across sequence lengths $T \in \{512, 1024, 2048, 4096, 8192\}$ and head dimensions $d \in \{64, 128\}$. We manually set the cache budget M to simulate different on-chip SRAM sizes, sweeping values from 32 KiB to 1 MiB. Note that M controls the tiled block size. Specifically, with M in bytes, we compute the row block size $|I|$ and column block size $|J|$ as: $|I| = \lfloor M/(16d) \rfloor, |J| = \min(|I|, d)$. For each (T, d), we select each method's configuration (threads, M, and, for masked attention, the scheduling strategy) by the median runtime on test sets 0–24 and report the median on test sets 25–49. We additionally profile the multi-threaded implementation on $T \in \{512, 8192\}$, $d \in \{64, 128\}$, and $M \in \{32\,\text{KiB}, 1\,\text{MiB}\}$ with 8 and 16 threads to break down the runtime of specific lines in Algorithm 1.

5. Experimental Results

In this section, we present our experimental results on runtime performance and the profiling analysis.

5.1. Runtime of Self-Attention

Figures 3a and 4a present the runtime and the speedup over SDPA for self-attention. For $T \geq 2048$, FA-2 is on par with SDPA (0.93–0.99×). For $T \leq 1024$, FA-2 is faster, by up to 1.60× at $T = 1024$. Compared with single-threaded standard attention, FA-2 reaches up to 13.8× speedup ($T = 8192$, $d = 64$, 16 threads).

5.2. Runtime of Masked Attention

Figures 3b and 4b present the runtime and the speedup over SDPA for masked attention. FA-2 is the fastest method at every T: it is 1.39–1.69× faster than SDPA for $T \geq 2048$, and up to 1.89× at $T = 1024$. The fastest configuration uses dynamic scheduling with 8 threads for $T \leq 1024$ and work-stealing with 16 threads for $T \geq 4096$. At $T = 2048$, both use 16 threads, with dynamic scheduling fastest at $d = 64$ and work-stealing at $d = 128$.

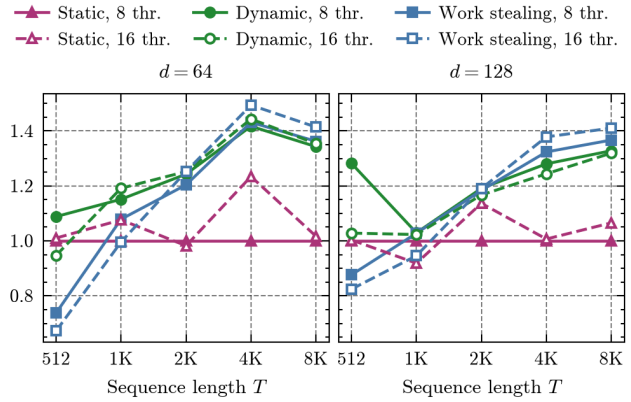
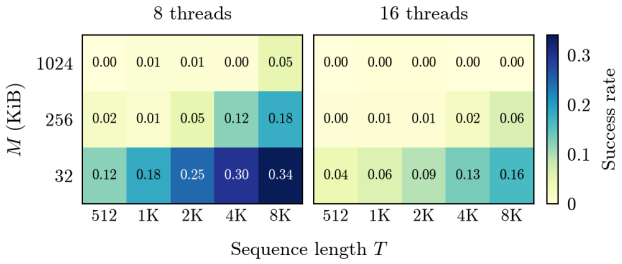
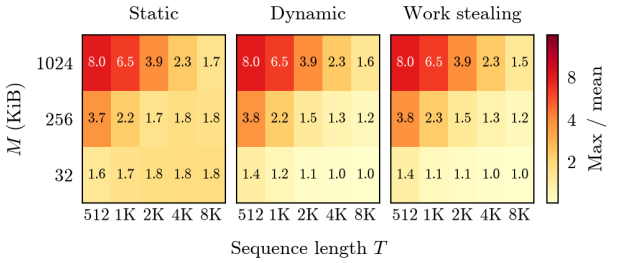


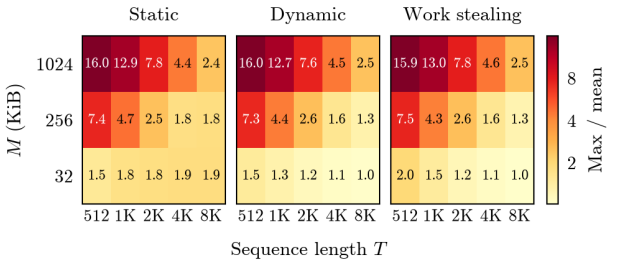
Figure 5. **Scheduling strategies for masked FA-2.** Speedup of each scheduling strategy over static scheduling with 8 threads. Each strategy uses its best M for each (T, d). Both balanced schedules outperform static scheduling for $T \geq 2048$, while work-stealing is slower than static at $T = 512$.



(a) Steal success rate for work-stealing.

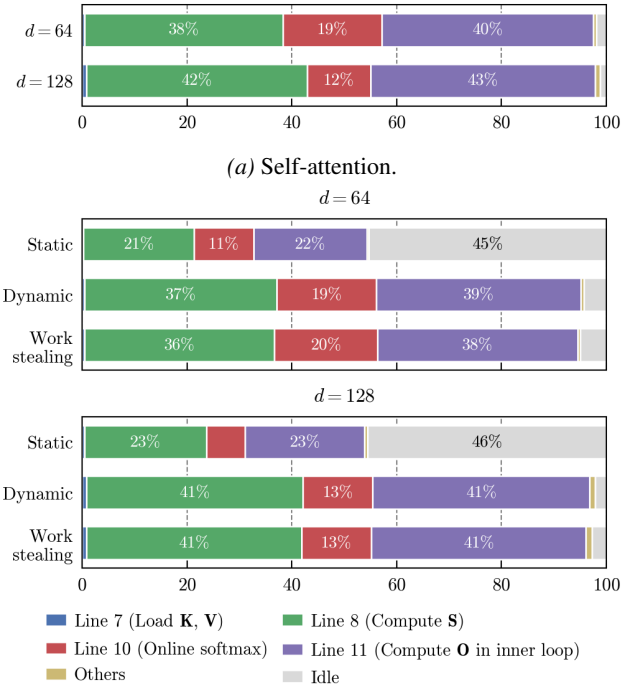


(b) Load imbalance, 8 threads.



(c) Load imbalance, 16 threads.

Figure 6. **Performance metrics of masked attention.** Values are averaged over $d \in \{64, 128\}$ and all test sets. With few row blocks (large M, small T), no schedule balances the load; with more row blocks, dynamic and work-stealing approach 1.0 while static stays near 1.8.



(a) Self-attention.

(b) Masked attention.

Figure 7. **Per-step profiling of the multi-threaded FA-2 forward kernel.** Bars show the share of total thread time (%) at $T = 8192$ and $M = 32$ KiB, averaged over 8 and 16 threads; idle is the time threads wait for the slowest thread. Static scheduling leaves 45–46% of thread time idle; dynamic and work-stealing leave 2–5%.

5.3. Scheduling Strategies for Masked Attention

Figure 5 compares the three scheduling strategies. For $T \geq 2048$ with 8 threads, dynamic and work-stealing are 1.18–1.43× faster than static. Between the two, dynamic is faster or tied for $T \leq 1024$, while work-stealing is slightly faster for $T \geq 4096$. At $T = 512$, work-stealing is slower than static (down to 0.67×), as there are too few blocks to amortize the cost of stealing. Relative to static scheduling with 8 threads, static scheduling with 16 threads achieves only 0.92–1.23×, whereas work-stealing with 16 threads is the fastest for $T \geq 4096$ (1.38–1.49×); the extra threads pay off mainly when the load is balanced.

Figure 6 explains these trends through the number of row blocks $T_r = \lceil T/|I| \rceil$. When T_r is small relative to the thread count, no schedule can balance the load. For example, at $M = 1$ MiB and $T = 512$, $T_r = 1$, so a single thread computes the whole output and the load imbalance equals the thread count under all three strategies. As T_r grows with larger T and smaller M, dynamic and work-stealing approach perfect balance, while static stays at 1.85. With p threads, static scheduling gives the last thread the most costly contiguous chunk of the triangle, $(2p - 1)/p^2$ of the work against a mean of $1/p$, so the imbalance is $(2p -$

$1)/p = 1.875$ at $p = 8$. Steals likewise succeed more often when blocks are plentiful: the steal success rate peaks at 0.34 at $T = 8192$ and $M = 32$ KiB.

5.4. Runtime Distribution of Algorithmic Steps

Figure 7 presents the runtime breakdown of the multi-threaded kernel at $T = 8192$ and $M = 32$ KiB. For self-attention, the matmuls in Lines 8 and 11 take most of the runtime, and Line 10 (Online softmax) takes 19% at $d = 64$ and 12% at $d = 128$. This is consistent with Sec. C.3: the matmul cost scales with d while the softmax cost does not. Line 7 (Load $\mathbf{K}, \mathbf{V}$) takes under 1% because our implementation accesses the $\mathbf{K}$ and $\mathbf{V}$ blocks through Eigen views without copying; their memory traffic is therefore counted within Lines 8 and 11. For masked attention, static scheduling leaves 45–46% of thread time idle, while dynamic and work-stealing leave 2–5%. The same model (Sec. 5.3) predicts $1 - p/(2p - 1) = 46.7\%$ idle at $p = 8$.

6. Discussion

Our study has several limitations. First, our implementations prioritize algorithmic transparency over maximal hardware utilization. We rely on Eigen's matrix products rather than hand-written SIMD kernels, so our results establish baseline algorithmic behavior rather than an upper bound for CPU inference. Second, we only test a single CPU architecture. Therefore, the phenomena observed in our experiments may not generalize to different CPU architectures (e.g., ARM). Finally, our evaluation utilized synthetic data, and it will be an interesting and important direction to further investigate the performance on real edge devices. Despite these limitations, this study serves as a critical preliminary exploration of the algorithm's behavior on CPU.

7. Conclusion

In this work, we evaluated the inference performance of the FA-2 forward kernel on CPU and benchmarked multi-threaded implementations against standard attention and SDPA. Multi-threaded FA-2 performs on par with SDPA for self-attention and is 1.39–1.69× faster for masked attention on long sequences. For masked attention, the scheduling strategy matters: static scheduling leaves 45% of thread time idle on the triangular workload, while dynamic and work-stealing scheduling reduce it to 2–5% and are up to 1.43× faster. Our observations underscore the importance of further investigation of efficient attention algorithms on CPU.

References

Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer, 2020. URL https://arxiv.org/abs/2004.05150.

Chen, B., Dao, T., Winsor, E., Song, Z., Rudra, A., and Ré, C. Scatterbrain: Unifying sparse and low-rank attention. In Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, 2021. URL https://openreview.net/forum?id=SehIKudiIo1.

Choromanski, K. M., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J. Q., Mohiuddin, A., Kaiser, L., Belanger, D. B., Colwell, L. J., and Weller, A. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=Ua6zuk0WRH.

Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=mZn2Xyh9Ec.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=H4DqfPSibmx.

Jiang, J., Du, J., Huang, D., Li, D., Zheng, J., and Lu, Y. Characterizing and optimizing transformer inference on arm many-core processor. In *Proceedings of the 51st International Conference on Parallel Processing*, ICPP '22, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450397339. doi: 10.1145/3545008.3545022. URL https://doi.org/10.1145/3545008.3545022.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are RNNs: Fast autoregressive transformers with linear attention. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 5156–5165. PMLR, 13–18 Jul 2020. URL https://proceedings.mlr.press/v119/katharopoulos20a.html.

Kitaev, N., Kaiser, L., and Levskaya, A. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=rkgNKkHtvB.

Mao, Y., Ester, M., and Li, K. Iceformer: Accelerated inference with long-sequence transformers on CPUs. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=6RR3wU4mSZ.

Roy, A., Saffar, M., Vaswani, A., and Grangier, D. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. doi: 10.1162/tacl_a_00353. URL https://aclanthology.org/2021.tacl-1.4/.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is all you need. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., and Ahmed, A. Big bird: Transformers for longer sequences. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 17283–17297. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/c8512d142a2d849725f31a9a7a361ab9-Paper.pdf.

Zhang, T., Yi, J., Yao, B., Xu, Z., and Shrivastava, A. Nomad-attention: Efficient llm inference on cpus through multiply-add-free attention. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 112706–112730. Curran Associates, Inc., 2024. doi: 10.52202/079017-3581. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ccda3c632cc8590ee60ca5ba226a4c30-Paper-Conference.pdf.

A. I/O and Computational Complexity of Standard Attention

For simplicity, we assume a simplified two-level memory hierarchy only consisting of on-chip SRAM and off-chip memory. In general, the input sequence length is much larger than the head dimension, i.e., $T \gg d$, and we may assume the intermediate $T \times T$ matrices $\mathbf{QK}^\top$ and $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})$ cannot be stored in on-chip SRAM.

We analyze the I/O and computational complexity of standard attention.

A.1. I/O Complexity

We consider the computations of $\mathbf{QK}^\top \in \mathbb{R}^{T\times T}$, $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d}) \in \mathbb{R}^{T\times T}$, and $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})\mathbf{V} \in \mathbb{R}^{T\times d}$ and the corresponding major memory accesses. To compute attention, we must sequentially:

- Write T^2 elements to off-chip memory to store $\mathbf{QK}^\top$.
- Read T^2 elements by loading $\mathbf{QK}^\top$ from off-chip memory to calculate $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})$, and write T^2 elements back to off-chip memory.
- Read T^2 elements by loading $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})$ from off-chip memory to calculate $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})\mathbf{V}$.

We conclude that standard attention induces $4T^2 = \mathcal{O}(T^2)$ memory accesses.

A.2. Computational Complexity

On the other hand, the computational complexity for each operation is, respectively,

- $\mathbf{QK}^\top$: $2(TdT) = 2dT^2$,
- $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})$: $4T^2$ (each row takes $4T$ operations for exponentiation, summation, and normalization),
- $\mathrm{softmax}(\mathbf{QK}^\top/\sqrt{d})\mathbf{V}$: $2(TdT) = 2dT^2$.

Summing up, standard attention induces $(4dT^2 + 4T^2) = \mathcal{O}(T^2d)$ operations.

B. Derivation of the FA-2 Forward Kernel

In this section, we present a derivation of the FA-2 forward kernel (Algorithm 1) and analyze its I/O and computational complexity.

B.1. Safe Softmax

We first review safe softmax, which is a numerically stable way to compute softmax. Recall that softmax exponentializes each element of a vector $\mathbf{z}$ and normalizes them by the sum of exponentials. This creates numerical instability when some elements of $\mathbf{z}$ are large, leading to overflow during exponentiation. For example, float32 can represent numbers up to approximately 3.4×10^{38}, which means $\exp(89) \approx 8.6 \times 10^{38}$ already overflows. To mitigate this, safe softmax simply subtracts the maximum element from $\mathbf{z}$ before exponentiation:

$$\frac{\exp(z_i)}{\sum_j \exp(z_j)} = \frac{\exp(z_i - \max_k z_k)}{\sum_j \exp(z_j - \max_k z_k)}. \tag{2}$$

Note that the equality holds trivially as $\exp(z_i - \max_k z_k) = \exp(z_i)/\exp(\max_k z_k)$.

B.2. Two-Pass Tiled Attention

We now present a derivation of the two-pass tiled attention algorithm, which is the basis for the FA-2 forward kernel. First, we partition $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ into the following row blocks:

$$\mathbf{Q} = \begin{bmatrix} Q_{I_1} \\ Q_{I_2} \\ \vdots \\ Q_{I_{T_r}} \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} K_{J_1} \\ K_{J_2} \\ \vdots \\ K_{J_{T_c}} \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} V_{J_1} \\ V_{J_2} \\ \vdots \\ V_{J_{T_c}} \end{bmatrix}$$

where $|I_i| = T/T_r =: |I|$ for all $i = 1, 2, \ldots, T_r$ and $|J_j| = T/T_c =: |J|$ for all $j = 1, 2, \ldots, T_c$. For each block I_i, we may rewrite attention as

$$\text{softmax}\left(\frac{\mathbf{Q}_{I_i}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} = \text{softmax}\left(\frac{1}{\sqrt{d}}\left[\mathbf{Q}_{I_i}\mathbf{K}_{J_1}^\top, \ldots, \mathbf{Q}_{I_i}\mathbf{K}_{J_{T_c}}^\top\right]\right)\begin{bmatrix} \mathbf{V}_{J_1} \\ \vdots \\ \mathbf{V}_{J_{T_c}} \end{bmatrix}. \tag{3}$$

Let $\mathbf{z}$ be a row of $\mathbf{Q}_{I_i}\mathbf{K}^\top$. Note that for $j = 1, 2, \ldots, T_c$, the following recurrence holds:

$$m_j = \max(m_{j-1}, \max_{t \in J_j} z_t) \tag{4}$$

$$d_j = d_{j-1} \cdot \exp(m_{j-1} - m_j) + \sum_{t \in J_j} \exp(z_t - m_j) \tag{5}$$

where $m_j = \max_{t \in J_1 \cup \cdots \cup J_j} z_t$ and $d_j = \sum_{t \in J_1 \cup \cdots \cup J_j} \exp(z_t - m_j)$. That is, we can actually compute the maximum and sum of exponentials iteratively over J_j. Using the above recurrence and simple matrix algebra, the tiled attention can be computed in two passes (Algorithm 2):

Algorithm 2 Two-Pass Tiled Attention

Divide $\mathbf{Q}$ into row-block matrices $\mathbf{Q}_{I_1}, \ldots \mathbf{Q}_{I_{T_r}}$ of size $|I| \times d$ and divide $\mathbf{K}$, $\mathbf{V}$ into $\mathbf{K}_{J_1}, \ldots, \mathbf{K}_{J_{T_c}}$ and $\mathbf{V}_{J_1}, \ldots, \mathbf{V}_{J_{T_c}}$ of size $|J| \times d$ each.
Divide the output $\mathbf{O}$ into row-block matrices $\mathbf{O}_{I_1}, \ldots \mathbf{O}_{I_{T_r}}$ of size $|I| \times d$ each.
for $i = 1$ to T_r **do**
  Load $\mathbf{Q}_{I_i}$ to on-chip SRAM.
  Initialize $\mathbf{O}_{I_i} \leftarrow \mathbf{0}_{|I_i| \times d}$, $\boldsymbol{\ell} \leftarrow \mathbf{0}_{|I_i|}$ and $\boldsymbol{m} \leftarrow (-\infty)_{|I_i|}$ on chip.
  for $j = 1$ to T_c **do**
    Load $\mathbf{K}_{J_j}$ to on-chip SRAM.
    Compute $\mathbf{S}_j \leftarrow \mathbf{Q}_{I_i}\mathbf{K}_{J_j}^\top$ on chip.
    Compute $\boldsymbol{m}_j \leftarrow \max(\boldsymbol{m}_{j-1}, \text{rowmax}(\mathbf{S}_j))$ on chip.
    # $\odot$ = component-wise product
    Compute $\boldsymbol{\ell}_j \leftarrow \exp(\boldsymbol{m}_{j-1} - \boldsymbol{m}_j) \odot \boldsymbol{\ell}_{j-1} + \text{rowsum}(\exp(\mathbf{S}_j - \boldsymbol{m}_j))$ on chip.
    Store $\mathbf{S}_j$ to off-chip memory.
  end for
  for $j = 1$ to T_c **do**
    Load $\mathbf{S}_j$ and $\mathbf{V}_{J_j}$ to on-chip SRAM.
    # $\oslash$ = component-wise division; $\odot$ = component-wise product
    Compute $\mathbf{P}_j \leftarrow (\mathbf{1} \oslash \boldsymbol{\ell}_{T_c}) \odot \exp(\mathbf{S}_j - \boldsymbol{m}_{T_c})$ on chip.
    Compute $\mathbf{O}_{I_i} \leftarrow \mathbf{O}_{I_i} + \mathbf{P}_j\mathbf{V}_{J_j}$ on chip.
  end for
  Store $\mathbf{O}_{I_i}$ to off-chip memory.
end for

B.3. One-Pass Online Attention

While the two-pass approach is an improvement, we can further fuse the passes into a single pass. Let $\mathbf{O}_{I_i}^{(j)} = \sum_{k=1}^{j} \exp(\mathbf{S}_k - \boldsymbol{m}_j)\mathbf{V}_{J_k}$. We observe the following relationship:

$$\mathbf{O}_{I_i}^{(j)} = \left[\sum_{k=1}^{j-1} \exp(\mathbf{S}_k - \boldsymbol{m}_j)\mathbf{V}_{J_k}\right] + \exp(\mathbf{S}_j - \boldsymbol{m}_j)\mathbf{V}_{J_j} \tag{6}$$

$$= \exp(\boldsymbol{m}_{j-1} - \boldsymbol{m}_j) \odot \left[\sum_{k=1}^{j-1} \exp(\mathbf{S}_k - \boldsymbol{m}_{j-1})\mathbf{V}_{J_k}\right] + \exp(\mathbf{S}_j - \boldsymbol{m}_j)\mathbf{V}_{J_j} \tag{7}$$

$$= \exp(\boldsymbol{m}_{j-1} - \boldsymbol{m}_j) \odot \mathbf{O}_{I_i}^{(j-1)} + \exp(\mathbf{S}_j - \boldsymbol{m}_j)\mathbf{V}_{J_j} \tag{8}$$

Now, one sees that

$$\mathbf{O}_{I_i} = (\mathbf{1} \oslash \boldsymbol{\ell}_{T_c}) \odot \mathbf{O}_{I_i}^{(T_c)}. \tag{9}$$

With some algebraic manipulations, we have successfully removed the forward dependency on $\boldsymbol{\ell}_{T_c}$ and $\boldsymbol{m}_{T_c}$. Now, we can fuse the two loops into one and arrive at the FA-2 forward kernel (Algorithm 1).

C. I/O and Computational Complexity of the FA-2 Forward Kernel

We analyze the I/O and computational complexity of the FA-2 forward kernel.

C.1. I/O Complexity

For each outer iteration i:

- We load $\mathbf{Q}_{I_i}$ and initialize $\mathbf{O}_{I_i}$ once. This costs $\mathcal{O}(|I|d)$.
- We iterate through the inner loop $j = 1, \ldots, T_c$ and load $\mathbf{K}_{J_j}$ and $\mathbf{V}_{J_j}$ for T_c times. This costs $\mathcal{O}((|J| \times T_c)d) = \mathcal{O}(Td)$.
- Finally, we write the result $\mathbf{O}_{I_i}$ back to off-chip memory. This costs $\mathcal{O}(|I|d)$.

Summing over the T_r outer iterations, the number of memory accesses is $\mathcal{O}(T_r \times Td) = \mathcal{O}(\frac{T^2 d}{|I|})$. Choosing $|I| = \Theta(M/d)$, we have total memory accesses $\mathcal{O}(\frac{T^2 d^2}{M})$. For typical values of T and d, this is much more I/O efficient.

C.2. Computational Complexity

For each (I_i, J_j), the matrix products $\mathbf{Q}_{I_i}\mathbf{K}_{J_j}^\top$ and $\mathbf{P}\mathbf{V}_{J_j}$ each costs $\mathcal{O}(|I||J|d)$. Summing over $T_r T_c = \frac{T}{|I|}\frac{T}{|J|}$ blocks, the computational complexity is $\mathcal{O}(T^2 d)$, which is the same as standard attention.

While the above analysis confirms that FA-2 achieves better memory complexity than standard attention, we also provide a finer-grained, line-by-line complexity analysis to accompany our runtime profiling in Sec. 5.4

C.3. Line-by-Line Complexity Analysis

C.3.1. Line-by-Line I/O Complexity

We calculate the exact number of elements transferred between SRAM and off-chip memory.

- **Line 4** (Load $\mathbf{Q}_{I_i}$ to on chip SRAM). Each block $\mathbf{Q}_{I_i}$ contains $|I|d$ elements. So, line 4 yields Td memory access cost.
- **Line 7** (Load $\mathbf{K}_{J_j}$ and $\mathbf{V}_{J_j}$ to on chip SRAM). Each (i, j) iteration reads $2|J|d$ elements. So, line 7 yields $2\frac{T^2 d}{|I|}$ memory access cost.
- **Line 15** (Store $\mathbf{O}_{I_i}$ to off-chip memory). Each block $\mathbf{O}_{I_i}$ contains $|I|d$ elements. So, line 15 yields Td memory access cost.

C.3.2. Line-by-Line Computational Complexity

Similarly, we break down the exact operation counts for matrix multiplication and online softmax.

- **Line 8** (Compute $\mathbf{S} \leftarrow \mathbf{Q}_{I_i}\mathbf{K}_{J_j}^\top$ on chip). Each (i, j) iteration costs $2|I||J|d$ operations. So, line 8 yields $2T^2d$ total computational cost.
- **Line 10** (Compute $\boldsymbol{m}_{\text{new}}$, $\mathbf{P}$ and $\boldsymbol{\ell}_{\text{new}}$ on chip). Note that $\text{rowmax}(\mathbf{S})$ requires $|I|(|J| - 1)$ operations. Updating $\boldsymbol{m}_{\text{new}}$ requires $|I|$ comparisons. Updating $\mathbf{P}$ requires $2|I||J|$ operations. Updating $\boldsymbol{\alpha} \odot \boldsymbol{\ell}$ requires $3|I|$ operations. Updating $\text{rowsum}(\mathbf{P})$ requires $|I|(|J| - 1)$ operations. Updating $\boldsymbol{\ell}_{\text{new}}$ requires additional $|I|$ additions. Summing up, line 10 requires $4|I||J| + 3|I|$ operations per execution. So, line 10 yields $4T^2 + 3\frac{T^2}{|J|}$ total computational cost.
- **Line 11** (Compute $\mathbf{O}_{I_i} \leftarrow \boldsymbol{\alpha} \odot \mathbf{O}_{I_i} + \mathbf{P}\mathbf{V}_{J_j}$ on chip). Updating $\boldsymbol{\alpha}$ requires $2|I|$ operations. Component-wise product with $\mathbf{O}_{I_i}$ requires $|I|d$ operations; $\mathbf{P}\mathbf{V}_{J_j}$ requires $2|I||J|d$ operations. Updating $\mathbf{O}_{I_i,:}$ requires additional $|I|d$ additions. Summing up, line 11 requires $2|I|(1 + d + |J|d)$ operations per execution. So, line 11 yields $2T^2d + 2T^2\frac{d}{|J|} + 2\frac{T^2}{|J|}$ total computational cost.
- **Line 14** (Compute $\mathbf{O}_{I_i} \leftarrow (\mathbf{1} \oslash \boldsymbol{\ell}) \odot \mathbf{O}_{I_i}$ on chip). Note that $\mathbf{1} \oslash \boldsymbol{\ell}$ requires $|I|$ operations. Component-wise product with $\mathbf{O}_{I_i}$ requires $|I|d$ operations. Summing up, line 14 requires $|I| + |I|d$ operations per execution. So, line 14 yields $Td + T$ total computational cost.